Ghid pentru descoperirea primului CVE

Creat de comunitatea ÎnvațăCyber



Versiunea 1.0 - 10.10.2026

Cuprins

Licență și drepturi de autor 3
Istoricul modificărilor 4
Versiunea inițiala 1.0 - 10.10.2026 4
Autori 5
Cuvânt înainte 6
Ce este un CVE? 7
Cum poti găsi primul CVE? 9
Privește perspectiva de securitate a aplicațiilor software folosite zilnic 10
Folosește o metodologie de identificare a vulnerabilităților sau creează-ti una 11
Utilizează puterea LLM-urilor în mod responsabil 12
Impactul unui CVE 13
Raportare responsabilă 14
Experiențe 15
Cristian Braneț 15
CVE-2025-49619 15
CVE-2025-62369 15
CVE-2025-63497 16
CVE-2026-48164, Other pending CVE 16
Concluzie 17

Licență și drepturi de autor

© 2026 Cristian Braneț. Toate drepturile rezervate.

Acest ghid are scop educațional și informativ și este destinat persoanelor care doresc să învețe procesul de identificare, analiză și raportare a vulnerabilităților informatice, în vederea obținerii primului CVE.

Condiții de utilizare

Conținutul original al acestui ghid poate fi copiat și distribuit **exclusiv în scopuri necomerciale**, cu condiția menționării autorului original și păstrării integrale a acestei notificări de licență.

Este strict interzisă utilizarea, comercializarea sau distribuirea acestui ghid în scopuri comerciale, inclusiv vânzarea, redistribuirea contra cost, includerea în produse sau servicii comerciale ori utilizarea sa ca material în cadrul unor activități comerciale.

Nicio prevedere a acestei notificări nu acordă dreptul de a revendica autoratul, de a elimina mențiunile privind drepturile de autor sau de a prezenta conținutul original drept creație proprie.

Identificatorii CVE, informațiile despre vulnerabilități, codul sursă și materialele provenite de la terți pot fi supuse unor drepturi și condiții de utilizare distincte.

Acest ghid nu garantează obținerea unui identificator CVE. Identificarea, testarea și raportarea vulnerabilităților trebuie realizate în mod responsabil și numai în sisteme pentru care există autorizare.

Istoricul modificărilor

Acest document a fost creat pe 10.10.2026. În continuare se află detalii despre modificările care au fost aduse de-a lungul timpului acestuia.

Versiunea inițiala 1.0 - 10.10.2026

Autori

Acest ghid a fost realizat de câțiva dintre membrii comunității ÎnvațăCyber, prezentați mai jos:

- Cristian Braneț - Junior Penetration Tester

Cuvânt înainte

Acest ghid a fost creat cu scopul de a ajuta persoanele la început de drum în domeniul securității cibernetice să afle ce este și cum se poate obține primul lor identificator CVE.

Ghidul conține modalități și sugestii pe care un practicant le poate folosi ca să cerceteze și să meargă în direcția obținerii primului CVE. Informațiile oferite pot fi ulterior integrate într-o metodologie de cercetare proprie deja existentă sau vor ajuta la crearea acesteia.

Obținerea unui identificator CVE este o activitate pe care un începător poate alege să o practice, deoarece îi va dezvolta garantat cunoștințe în domeniul securității cibernetice, precum:

- **Dezvoltarea unei metodologii de testare a aplicațiilor web, binare, etc.;**
- **Dezvoltarea abilităților de raportare scrisă a problemelor de securitate în aplicații software sau hardware;**
- **Raportare responsabilă și coordonată a problemelor de securitate cibernetică către persoanele responsabile;**
- **Analiza de cod sursă în scopul identificării vulnerabilităților;**
- **Crearea unui mediu controlat de testare a software-ului;**
- **Dezvoltarea abilităților de prompting pentru LLM-uri, în cazul în care acestea sunt folosite pentru realizarea cercetării.**

Punctele prezentate mai sus sunt doar o parte din felul în care o persoană își poate dezvolta cunoștințele în domeniu prin obținerea unui identificator CVE.

În afară de experiența educațională, cercetătorii, indiferent de nivelul de experiență, au și alte motivații pentru a investi timp în direcția obținerii de identificatoare CVE, precum:

- **Dezvoltarea reputației de specialist în domeniul securității cibernetice** – ceea ce facilitează ulterior obținerea de job-uri în domeniu, de exemplu;
- **Îmbunătățirea securității programelor software open-source sau proprietare** – aceste programe au mulți utilizatori și pot fi vizate de atacatori, dar prin identificarea unei probleme de securitate și asignarea unui identificator CVE către aceasta, problema este reparată de dezvoltatori și căile de atac sunt reduse.

Această activitate în domeniul securității cibernetice aduce doar beneficii pentru cercetători, comunitate și pentru peisajul cibernetic global. Din acest motiv, considerăm că este o activitate căreia orice persoană din domeniul securității cibernetice merită la un moment dat în cariera lor să îi acorde niște timp pentru a o realiza.

Ce este un CVE?

Programul Common Vulnerabilities & Exposures (abreviat CVE) reprezintă un catalog al tuturor problemelor de securitate și greșelilor de configurare întalnite în programe software și în hardware.

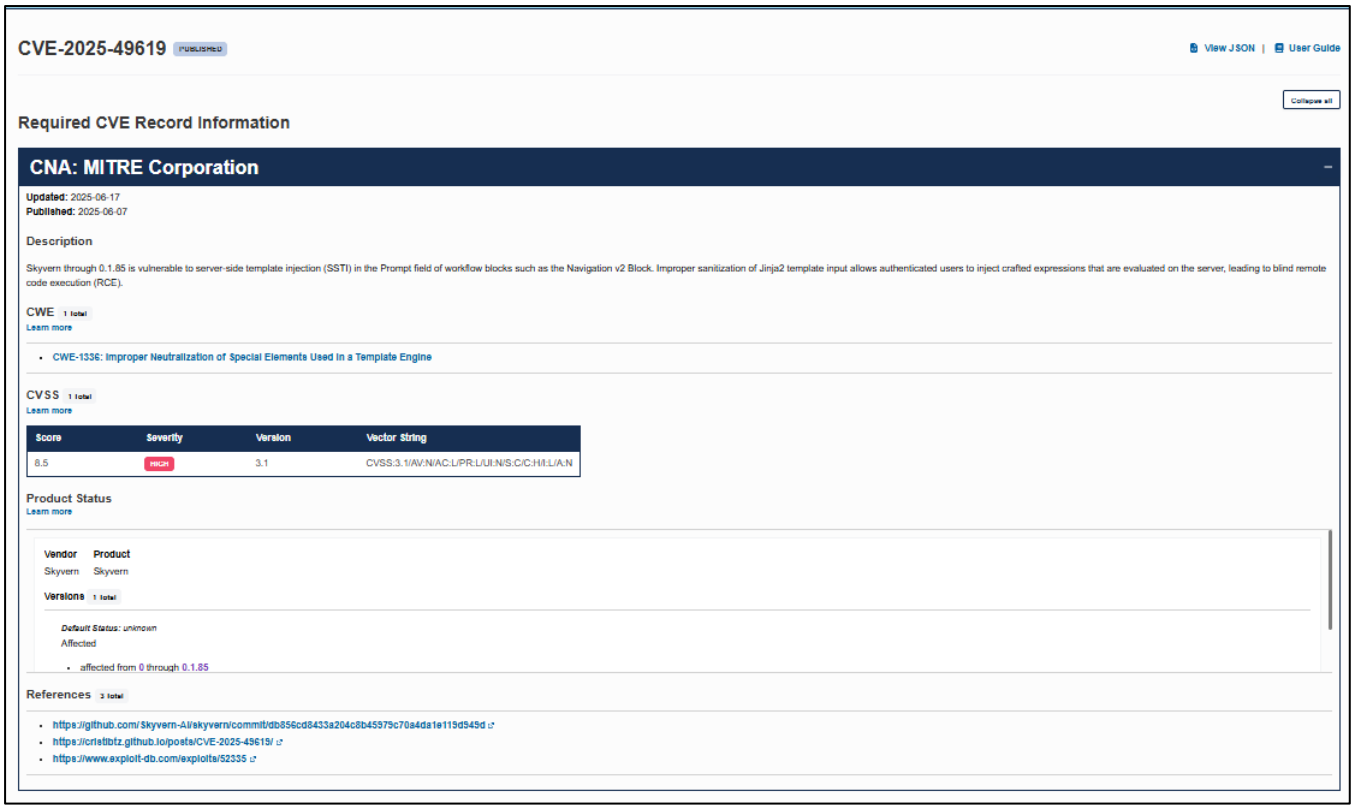
A fost creat în 1999 și este operat de MITRE Corporation cu susținerea Departamentului de Securitate Internă al Statelor Unite.

Programul funcționează prin asignarea de identificatori CVE unici fiecărei vulnerabilități descoperite în programe software sau în hardware. Identificatorii sunt de forma **CVE-AN_RAPORTARE-COD_UNIC_ASIGNAT**.

Scopul programului CVE este de a documenta într-un loc centralizat toate vulnerabilitățile dezvăluite, cum am menționat și mai sus. ***Dar care este utilitatea în a face acest demers?*** Utilitatea este faptul că o vulnerabilitate este scoasă la suprafață, fapt care îi va face pe dezvoltatori să o repare. Un atacator care știa de aceasta și încă nu a folosit-o, îi pierde complet utilitatea, mai ales dupa ce utilizatorii software-ului actualizează la versiunea sigură și problema este dezvăluită public și ușor de identificat prin codul asignat. O altă utilitate este că CVE permite o mapare între un cod unic și software, versiune vulnerabilă si detaliile vulnerabilității. Acest lucru ajută în cadrul testelor de penetrare sau în scanările de securitate pentru a identifica rapid unele probleme de securitate rămase nereparate într-o organizație.

Există autorități CNA (CVE Numbering Authorities) care au sarcina de a analiza vulnerabilitățile raportate de către cercetători și să le asigneze identificator CVE daca se consideră potrivite. Câteva exemple de CNA sunt: MITRE, Github, VulDB, etc. Deși toate CNA-urile funcționează după aceleași reguli la bază, se întâmplă să existe și diferențe, de exemplu: anumite CNA-uri pot răspunde la cereri și asigna CVE-uri considerabil mai rapid decat altele; unele CNA-uri vor asigna CVE la o anumită vulnerabilitate, iar altele poate nu vor asigna, etc.

Un CVE conține informații despre vulnerabilitatea raportată, precum: Data publicării, Descriere, Scor CVSS, CWE și Referințe.



Figură 1 Exemplu de înregistrare CVE

Data publicării reprezintă ziua, luna și anul când vulnerabilitatea a fost publicată, NU descoperită sau reparată. Descoperirea și repararea au loc înaintea publicării!

CWE (Common Weakness Enumeration) reprezintă un sistem de categorizare a problemelor de securitate, o descriere a lor la modul general.

Scorul CVSS (Common Vulnerability Scoring System) este un standard folosit pentru a măsura severitatea unei probleme de securitate.

Referințele reprezintă prima formă de dezvăluire publică a vulnerabilității, o dovadă că aceasta există sau a existat. În momentul în care cerem un CVE este nevoie să trimitem și o referință.

Aceasta este o prezentare pe scurt a ce este CVE. În următoarele versiuni ale ghidului vom merge mai în profunzime. Recomandăm citirea următoarelor resurse pentru a înțelege mai bine CVE:

- https://www.cve.org/
- https://cwe.mitre.org/
- https://en.wikipedia.org/wiki/Common_Vulnerability_Scoring_System
- https://github.com/advisories

Cum poti găsi primul CVE?

Găsirea primului CVE poate avea diverse forme:

- poți descoperi întâmplător o vulnerabilitate într-o aplicație pe care o folosești în mod normal;
- poți să planuiești să faci cercetare pentru obținere de CVE și în urma unei acesteia să identifici vulnerabilități – această cercetare poate fi cu mai multe persoane și să fiți creditați toți pentru descoperire;
- te poți afla în cadrul unei testări de penetrare și identifici probleme de securitate din software-ul open source folosit de client, raportându-le acestuia, dar și dezvoltatorilor software-ului;
- utilizarea LLM-urilor pentru a descoperi vulnerabilități și a le repara.

Privește perspectiva de securitate a aplicațiilor software folosite zilnic

Una dintre modalitățile prin care se poate obține un CVE este atunci când analizezi implicațiile de securitate ale unei aplicații și încerci să folosești funcționalitățile ei într-un mod neintenționat inițial. Până la urmă aceasta este una din acțiunile de bază pe care să le faci într-o testare de penetrare și un mod de gândire esențial în acest domeniu.

Deși poate aplicația este open source și ai acces la codul sursă, poate fi mai distractiv să o privești din perspectivă black box și să te întrebi:

- Ce limbaj de programare folosește aplicația și ce tipuri de vulnerabilități pot apărea în acest limbaj?
- Ce se întamplă daca trimit date malformate către un endpoint?
- Cum circulă datele în aplicație?
- Cum sunt afișate datele?

Vă puteți pune multe întrebări de acest fel, dar cel mai important este să lansați aplicația și să o folosiți într-un mod complet diferit de cum o folosiți în viața de zi cu zi și, vom spune cu puțină ironie, „să vedeți unde crapă”.

Folosește o metodologie de identificare a vulnerabilităților sau creează-ți una

O metodologie reprezintă un set de pași pe care îi urmezi în realizarea unui scop. Considerăm că o metodologie trebuie să fie deterministică. Ar fi enervant să urmezi același set de pași și să ai mereu rezultate diferite.

Un exemplu de metodologie ar fi daca ai alege să cauți probleme de securitate doar în aplicații PHP și să folosești functia „grep" pe codul sursa pentru a identifica secvențe de cod ce indică o posibilă problemă de securitate, apoi să analizezi acel cod raportat la contextul în care se află, în funcție de asta să presupui dacă există o vulnerabilitate, să lansezi aplicația și să validezi și apoi să raportezi vulnerabilitatea sau dacă nu există niciuna, să treci la altă aplicație și să repeți pașii de dinainte.

Un articol util care merită citit și expune o modalitate de a cauta CVE-uri similară cu pașii de mai sus este următorul: https://medium.com/@dub-flow/the-easiest-way-to-find-cves-at-the-moment-github-dorks-29d18b0c6900. Autorul se folosește de Github Dorks pentru a identifica anumite secvențe de cod și abia apoi alege să continue evaluarea aplicației și găsește vulnerabilități. O metodă excelentă.

Utilizează puterea LLM-urilor în mod responsabil

În 2026, avem acces la LLM-uri care pot ușura considerabil procesul de căutare de vulnerabilități în cod sursă. Până la urmă este specialitatea lor să proceseze text si să ruleze „grep”, dar de multe ori este nevoie de validare umană, pentru că dacă prompt-ul inițial nu este optim, AI-ul poate să nu ofere cele mai bune rezultate. Cu toate acestea, anumite modele cu prompt-uri potrivite deja nici nu mai au nevoie de validare umană. Pur și simplu analizează codul sursă, identifică problema, rulează aplicația și chiar raportează.

Unii oameni pot fi sceptici legat de folosirea LLM-urilor pentru a cauta CVE-uri, deoarece au dat posibilitatea multor persoane care nu au cu adevărat nivelul tehnic să facă asta și nici să valideze cum trebuie rezultatele LLM-urilor. Din acest motiv, multe raportări sunt ”aberante” și doar încetinesc procesul de triere și fac vulnerabilitățile reale să fie analizate mult mai târziu decat ar trebui.

Totuși, nu suntem împotriva folosirii LLM-urilor pentru a cauta CVE-uri, ci o încurajăm, doar DACĂ există validare umană și un nivel tehnic minim de a întelege problemele de securitate ce pot apărea în aplicațiile evaluate (web, mobile, binare, etc.). Folosirea corectă a LLM-urilor poate accelera considerabil procesul de descoperire a vulnerabilităților și obținere a unui CVE, iar dacă ne folosim de el și pentru a repara problema, apoi să o validăm ca oameni, vom ajuta considerabil dezvoltatorii și persoanele care se ocupă de triere.

Din experiența noastră, am observat că un LLM se pricepe foarte bine să identifice vulnerabilitățile ce țin de injecții (XSS, SQLi, Command Injection, SSTI, etc.), deoarece aceste vulnerabilități sunt identificabile din tipare de cod pe care chiar și un LLM promptuit ineficient le poate identifica. Un prompt prost, precum „Find SSTI” dat unui LLM poate duce la identificarea acestei vulnerabilități, mai ales daca este folosit pe mai multe repozitoare open source.

În secțiunea Experiențe vom prezenta și exemple.

Impactul unui CVE

Indiferent de modul în care se obține un CVE, considerăm că este important ca acesta să aiba un impact pozitiv și real asupra unor utilizatori. În prezent, un CVE poate fi obținut chiar și pentru o vulnerabilitate într-o aplicație cu 50-100 de stele pe Github, de exemplu, pe care nu o folosește nimeni. În acest caz, acest CVE este doar un cod fară nicio valoare, deoarece e posibil ca aplicația să nu aibă o bază de utilizatori. Este un CVE artificial.

Considerăm mai importantă, evaluarea de securitate a proiectelor open source cu utilizatori reali. Nu vom da exemple, deoarece sunt foarte multe și se pot găsi ușor. De multe ori, numărul de stele nu denotă numărul real al utilizatorilor. Depinde de cercetător să afle prin diverse modalități dacă aplicația chiar este folosită, de câți oameni este folosită și dacă poate fi ținta atacurilor.

O dată cu apariția LLM-urilor și a modului în care au ușurat găsirea de probleme de securitate, credem că valoarea pe care o au anumite CVE-uri poate varia considerabil. De exemplu, un CVE în server-ul web Apache este mult mai valoros decât unul găsit într-un CMS abia lansat. Server-ul Apache are foarte mulți utilizatori și o postură de securitate solidă, ceea ce face chiar și analiza de securitate a acestuia cu un LLM bine promptuit destul de dificilă, pe când într-un CMS scris în PHP abia lansat este posibil ca un LLM prost promptuit să identifice probleme legitime de securitate în cateva minute.

Auditarea unor proiecte open source precum Apache necesită cunoștințe tehnice bine cimentate din partea unui cercetător pentru a găsi probleme de securitate chiar și folosind un LLM, dar măcar acel CVE are valoare reală și impact real!

Raportare responsabilă și coordonată

O parte foarte importantă înainte să facem o evaluare de securitate pe orice aplicație este să înțelegem regulile impuse de dezvoltatori referitor la politica lor de asignare a CVE-urilor (asta dacă sunt în măsură să asigneze), ce vulnerabilități acceptă să le considere probleme reale de securitate, care sunt resursele la care permit testarea de securitate și care este modul în care se coordonează cu cercetătorii pentru o raportare și publicare responsabilă a vulnerabilităților.

Există o politică standard în domeniu care oferă timp de 90 de zile dezvoltatorilor să repare și să dezvăluie vulnerabilitățile care le sunt raportate. Dacă aceste 90 de zile sunt depășite, cercetătorul poate să faca publică vulnerabilitatea individual. Este foarte important să aflam politica de reparare și publicare a vulnerabilităților definită de dezvoltatorii a căror aplicație o testăm. Recomandăm să colaborați cu răbdare și înțelegere, deoarece scopul este să ajutăm, nu să încurcăm dezvoltatorii.

Următorul articol prezintă această politică în mod general și recomandăm citirea lui: https://en.wikipedia.org/wiki/Coordinated_vulnerability_disclosure.

Experiențe

În continuare, vom prezenta experiențe pe care membrii comunității ÎnvațăCyber le-au avut în căutarea și descoperirea de CVE-uri.

Cristian Braneț

CVE-2025-49619

Acest CVE identifică o vulnerabilitate de tip Server Side Template Injection în aplicația de automatizări în browser folosind LLM-uri numită Skyvern. Am aflat de această aplicație, deoarece a trebuit la primul meu loc de munca să folosesc această aplicație, să înțeleg cum merge și să aduc contribuții la codul ei sursă.

Nu plănuisem să gasesc vreo problemă de securitate în ea, dar cu toate acestea a devenit primul meu CVE.

A durat o zi întreagă să o deployez, deoarece încă învățam multe lucruri despre DevOps și modul de deployment al aplicației mă luase prin surprindere, deși acum scriind aici consider că nu era ceva foarte greu.

Dupa deployment, am început să o folosesc ca un utilizator normal pentru a îmi da seama cum pot contribui la ea. Observasem că este scrisă în Python și știam că în aplicațiile web care folosesc Python poate apărea vulnerabilitatea Server Side Template Injection.

Mi-a luat o săptămână să mă gandesc să încerc un payload pentru a vedea dacă rezultă vreun comportament neobișnuit în aplicație, deși îi folosisem in mod legitim input-ul vulnerabil de sute de ori. Am fost șocat să văd că aplicația are un comportament neașteptat la un payload de SSTI. Nu îmi venea să cred, căci eram pe cale să găsesc prima mea vulnerabilitate.

Am testat continuu problema, i-am creat „Proof Of Concept”, am validat-o și pe versiunea cloud a aplicației, care era aproape identică cu cea open source și apoi am raportat către dezvoltatori și către MITRE.

Cum s-a prezentat și in paginile de dinainte, am descoperit problema prin folosirea legitima a aplicației și eventual am vrut să evaluez și postura de securitate.

CVE-2025-62369

Acest CVE este tot un SSTI, dar de data asta într-o aplicație PHP, numită Xibo CMS. Fiind fascinat de SSTI după descoperirea primului meu CVE, am decis să găsesc un SSTI și în PHP.

Am căutat pe Internet, secvențe de cod care să ducă în direcția de SSTI în aplicații care foloseau librăria Twig. Am integrat aceste secvențe în interogări Github Dorks și analizând sutele de rezultate manual, s-a întamplat să identific aplicația Xibo CMS, care folosea template-uri destul de mult.

Aici a fost prima oară când am folosit un LLM și i-am cerut să urmeărească felul cum circulă datele ca să ajungă în funcția vulnerabilă. Acesta nu a reușit să urmeze cu succes felul cum circulă datele, dar mi-a oferit niște observații valoroase care m-au dus pe drumul cel bun în direcția validării manuale și raportării vulnerabilității.

CVE-2025-63497

Acest CVE a fost mai mult un experiment de al meu pentru a vedea cu câtă ușurință se poate obține un CVE. Nu prea sunt interesat de el și nici nu îl consider important. Proiectul avea în jur de 100 de stele, era un CMS cu tema medicala, părea mai mult proiect personal. Reprezintă o vulnerabilitate de tip injecție SQL.

Am identificat problema folosind Github Dorks-uri pentru tipare de cod vulnerabile la injecție SQL în repozitoare de PHP cu mai puțin de 100-200 de stele.

CVE-2026-48164, Other pending CVE

Aceste doua CVE-uri au fost ambele obținute doar prin utilizarea unui LLM pentru analiza codului sursă și apoi doar am validat manual că vulnerabilitatea există.

CVE-2026-48164 este un HTML injection în Collabora.

Celalalt CVE care încă nu a fost publicat este un Path Traversal găsit în 15 minute în urma unui prompt slab de forma: „You are a web app security researcher. Please look for injection flaws in this codebase, like: RCE, SSTI, SQLi, XSS, LFI, Path Traversal, etc. Take into consideration OWASP Top 10 vulnerabilities.”. Spre surprinderea mea, a dat rezultate.

Concluzie

Există multe modalități în prezent de a obține un CVE. Credem că este important ca un CVE să fie obținut pentru aplicații care contează și care au utilizatori, indiferent dacă cercetătorul se folosește responsabil de un LLM, îl găsește întâmplător sau folosește o metodologie.

Este o activitate benefică pe care o încurajăm pentru persoanele pasionate de securitate cibernetică, care au deja niște cunoștințe de bază în IT și securitate cibernetică.

Acest ghid este în continuă dezvoltare și încurajăm pe oricine are experiență cu CVE-uri să contribuie cu sfaturi și să povestească despre experiența lui în a găsi vulnerabilități și a le raporta responsabil. Fiecare contributor va primi creditele meritate.